SHARE: Sustainable Heterogeneous Architectures can Reduce Emissions by Sharing Memory

Jack Toubes
Electrical Engineering and Computer Science
UC Berkeley
Berkeley, California, United States
jtoubes@berkeley.edu

Stefanos Kaxiras
Information Technology
Uppsala University
Uppsala, Sweden
stefanos.kaxiras@it.uu.se

Margaret Martonosi
Computer Science
Princeton University
Princeton, New Jersey, United States
mrm@princeton.edu

***Abstract*—Though hardware accelerators decrease the operational carbon emissions of contemporary systems-on-chip (SoCs), their significant area, and thus embodied carbon cost, makes their impact on the overall lifetime sustainability of a chip less clear. As opposed to instantiating separate hardware for each accelerated application kernel, recent publications have suggested time-sharing reconfigurable fabrics like CGRAs for multiple kernels, to decrease the total chip area required for acceleration at the cost of increased energy consumption. Motivated by the large overheads of reconfigurable fabrics, and the high area consumption of memory in hardware accelerators, in this paper we evaluate the carbon efficiency of sharing memory while retaining fixed-function compute hardware, as opposed to reconfigurable fabrics, and to fixed-function compute with unshared memory. We show that this strategy is more sustainable than using a reconfigurable fabric.**

I. Introduction

Information and Communication Technology (ICT) is responsible for between 2.1% and 3.9% of all greenhouse gas emissions globally [1]. Current trends threaten to increase these percentages significantly. These include the growth in AI and datacenter construction [2], [3], as well as an expected uptick with the roll-out of autonomous vehicles [4]. Considering these circumstances, even optimistic perspectives on ICT's carbon footprint believe that a call to action is necessary [5].

Processors and systems-on-chip (SoCs) are a significant contributor to ICT's emissions. For example, Apple's A17 Pro SoC contributes between 14% and 21% of the iPhone 15 Pro's total emissions [6], [7].[1] In Microsoft's datacenters, despite massive amounts of storage and memory, processors are responsible for around 14% of carbon emissions [8].

The carbon footprint of an SoC can be credited to two main sources. The first is a processor's *operational emissions*, the result of its power/energy consumption. The second is the processor's *embodied emissions*, which account for the carbon footprint of fabricating, and to a lesser extent transporting and disposing of the processor. Optimizing for operational emissions has historically been the focus of power-efficient and sustainable computing [9], but this is no longer sufficient [10]. Returning to the example from before, embodied emissions are responsible for between 48% and 73% of the Apple A17 Pro SoC's total emissions [6], [7].[1]

Although a chip's embodied emissions are highly dependent on the technology node in which it is fabricated, holding technology node constant leads to approximately constant embodied emissions per wafer [11], [12]. This means that architects can optimize for embodied emissions by decreasing a processor's area, such that we get more chips out of a wafer.

Nowadays, hardware acceleration is often the biggest contributor to an SoC's area—as long ago as 2018, fixed-function, domain-specific accelerators (DSAs) already took up over two-thirds of the total silicon area on state-of-the-art smartphone SoCs [13]. These accelerators increase embodied emissions while decreasing the operational emissions through more energy efficient processing. This is especially true for fixed-function DSAs as each DSA requires its own distinct, low-utilization silicon area [14], [15].

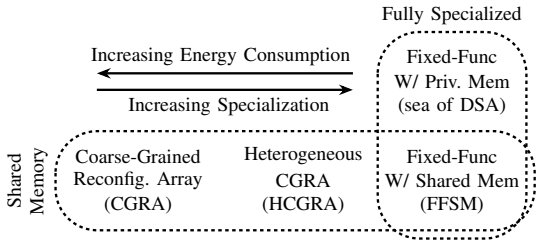


Fig. 1. The explored architectures, including trends in compute specialization and power consumption, and which architectures share memory between multiple kernels.

Recent works have argued that acceleration using reconfigurable fabrics, like FPGAs and CGRAs, may be more sustainable than the hegemonic sea of DSAs architecture for contemporary SoCs. As early as 2018, it was argued that the low-utilization silicon introduced for sea of DSAs architectures was not sustainable when considering it's embodied emissions overhead [14]. Later work built off of this, suggesting that reconfigurable fabrics use less silicon area than a sea of DSAs, since their area can be time-shared by many kernels; they argue that this reduces embodied emissions enough to make reconfigurable acceleration more sustainable than a sea of DSAs [11]. Finally, [16] argued this point further, using Aladdin [17] for DSA power and area estimates for eight kernels, and using these estimates to compare with the same kernels running on a CGRA. They found that CGRA could achieve a sustainability victory by replacing as few as three DSAs in some cases.

However, we believe that this existing work has missed some important insights. Specifically, their narrow focus on modifications to compute hardware mean they don't thoroughly consider the effects of on-chip memory. This is despite the fact that a DSA's private-local memory, be it scratchpad or cache, regularly "occupies most of the accelerator area" [18], 40%-90% in one study [19].

When using a reconfigurable fabric, kernels time-share compute *and memory*. Both compute and memory sharing theoretically have the potential to reduce total area, but they require significantly different concessions. Sharing compute requires replacing purpose-built hardware with much larger, slower, low-utilization, reconfigurable compute elements. On the other hand, in many cases sharing memory just requires the ability to time-multiplex access to the memory ports. This is to say that the overhead for sharing compute is significantly

[1]Range based on the SoC consuming between 25% and 75% of the iPhone's energy.

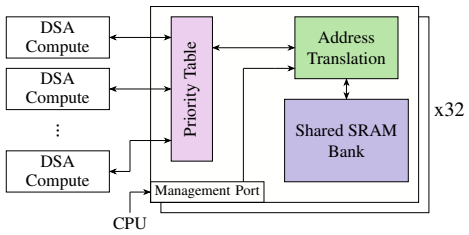


Fig. 2. The hardware, inspired by an Accelerator Store [19], used by our FFSM architecture for sharing memory between the compute hardware for multiple, fixed-function DSAs

larger than the overhead for sharing memory.

Combined, these two facts (reconfigurable compute has massive overheads and memory is a big area consumer) motivate sharing just memory, while retaining fixed-function compute, as a method for sustainable acceleration. Specifically, a **F**ixed-**F**unction compute with **S**hared **M**emory (FFSM) architecture would be expected to beat reconfigurable compute in power consumption by a large margin, with the potential to match or beat a reconfigurable fabric's area. FFSM can also provide a substantial area decrease while nearly matching the power and performance of fixed-function compute with *unshared* memory ("sea of DSAs" in this paper).

In this work we explore how carbon-efficient this FFSM architecture is relative to those proposed in the aforementioned recent works. To do this, we implemented a representative example of this FFSM architecture, inspired by the Accelerator Store [19]. As shown in Figure 2, this architecture uses low-overhead hardware for sharing a pool of SRAM banks amongst multiple DSAs. As summarized in Figure 1, we compare this FFSM architecture to two variants of CGRA, and to DSAs with unshared, private-local memories.

We compiled, generated, and synthesized hardware descriptions of these different architectures to acquire realistic power and area estimates. These power and area estimates were used to perform a first order carbon emissions analysis of each architecture with a variety of different kernel memory requirements and relative weights of operational and embodied emissions.

Our major contributions are as follows:

- We show that past work focused on sharing compute hardware to decrease an SoC's carbon footprint has misattributed the benefits of sharing memory to using reconfigurable compute hardware. When ignoring memory, to be more sustainable, a single 8x8 CGRA (HCGRA) would have to replace the compute of 109 (68) fixed-function DSAs, which is far more kernels than can share compute without performance degredation due to timing conflicts.
- There are no explored conditions where FFSM is definitively less sustainable than HCGRA/CGRA.
- In the vast majority of explored cases, an FFSM architecture is more sustainable than a sea of DSAs even when shared by only a small number of kernels (less than 6 kernels in all but the most operationally dominated cases).
- When embodied emissions dominate and reconfigurable compute cannot extend a device's lifespan (like in mobile/embedded computing domains) FFSM can reduce emissions compared to a baseline sea of DSAs when shared by up to 6x (4x) less kernels than CGRA (HCGRA).
- When operational emissions dominate and reconfigurability may extend lifetimes (more likely in the datacenter) FFSM can reduce emissions compared to a baseline sea of DSAs when shared by up to 9x (5x) less kernels than CGRA (HCGRA).

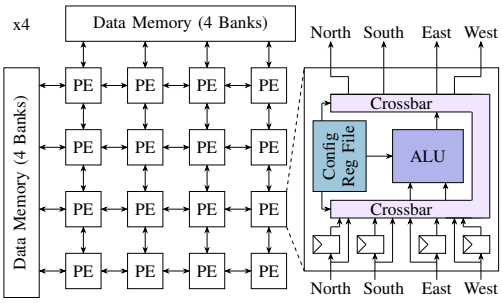


Fig. 3. The CGRA, based on HyCube [20], analyzed in this work. We modeled an 8x8 CGRA using 4, 4x4 CGRAs.

- We show that FFSM's sustainability scales well with increased compute performance, and demonstrate one way to cluster kernels together for sharing to minimize area.

II. Background

A. Processing Model

We assume for the purpose of this study that, if a CGRA is being considered as a viable option, then the performance of a CGRA is sufficient for a given SoC's applications. Furthermore, in order to perform a fair carbon comparison, we consider architectures with equivalent performance characteristics. For this reason, the DSAs we consider are necessarily small and simple. Though production DSAs vary greatly in size and complexity, a DSA whose performance matches that of a CGRA will generally be quite small in comparison, as the throughput of a CGRA is low compared to what a comparably sized DSA can achieve.

We model each accelerator as connected directly to a lowest-level memory, which can be shared (CGRA/HCGRA/FFSM) or unshared (sea of DSA). This lowest level memory is then connected to a *shared* higher level memory (e.g. shared L2, LLC, or main memory). Since this higher level of memory is always shared, we do not include it in our quantitative analysis. We leave analysis of more complex memory sharing hierarchies (e.g. partially shared L2 caches) for future work.

B. Considered Accelerator Architectures

1) Coarse grained reconfigurable array (CGRA): A Coarse-grained reconfigurable array, or CGRA, is a dataflow architecture consisting of a reconfigurable grid of processing elements, as shown in Figure 3. Unlike FPGAs, data moves through the CGRA in entire words. Thus, bistreams are smaller and CGRAs can be reconfigured faster than FPGAs, better for time-sharing by many kernels. CGRAs also have lower power and area overheads compared to FPGAs [20].

2) Heterogeneous CGRA (HCGRA): A traditional CGRA's PEs all can perform the same operations. If some of those operations are rarely or never used by the CGRA, then some instances of the operation were instantiated unnecessarily. Similarly, and more significantly, as highlighted in [21], the configuration register files inside of those PEs may not need to be able to represent all configurations if some are highly unlikely or the configuration changes infrequently. By removing unneeded hardware, *heterogeneous CGRAs* capitalize on these and similar insights to reduce area and power consumption of a CGRA by factors of 30% to 50% with little to no performance cost in some applications [21]–[23].

3) Sea of Domain-Specific Accelerators (DSAs): Fixed-function DSAs consist of specialized, purpose-built hardware designed to perform a particular, repeated computational task with high performance and power efficiency. Modern day SoCs often contain many tens of DSAs—a sea of DSAs [14], [17]. Because a DSA can be fine tuned to

the algorithm it is performing, it will have a smaller area and power consumption than a reconfigurable fabric configured to execute the same algorithm, however to perform many distinct operations, you will need to instantiate many distinct DSAs.

4) Shared Memory: We implemented in RTL a representative example of an architecture which shares memory amongst a group of fixed-function accelerators, as seen in Figure 2. Inspired by the Accelerator Store [19], our design uses priority tables for selecting which accelerator is allowed to access a given memory bank, and an address translation layer which can be configured to use an SRAM bank as a FIFO buffer or for random access. Unlike the Accelerator Store, each of an accelerator's memory access ports connects to only a single SRAM bank, a strategy which scales better than Accelerator Store's shared routing scheme connecting all ports to all banks.

III. Methodology

A. Analyzing a Processor's Carbon Footprint

Multiple recent publications have proposed architectural models for estimating a processor's carbon footprint [9], [11], [24]. Each of these model operational emissions with power (or energy) consumption (P) multiplied by carbon emitted per unit of power (or energy) (C_p). Embodied emissions are modeled by chip area (A) multiplied by the carbon emitted per unit area during manufacturing (C_a).

$$CF_{total} = CF_{embodied} + CF_{operational} = C_a A + C_p P \quad (1)$$

Different models take different approaches to filling in the values for C_a and C_p. FOCAL [11], the model we use in this paper, fills in C_a and C_p according to equation 2.

$$C_a = \frac{\alpha_{E2O}}{A_{ref}} \qquad C_p = \frac{(1-\alpha_{E2O})}{P_{ref}} \quad (2)$$

A_{ref} and P_{ref} are the area and power consumption for a reference architecture, such that CF_{total} for the reference will evaluate to 1. The operating condition, α_{E2O}, is a hyperparameter between 0 and 1 representing the relative proportion of the total carbon footprint attributed to embodied emissions for the *reference architecture* (architectures with different areas and power consumptions will necessarily have different operating conditions).

We use FOCAL since it's parameterization allows for easily examining different operating conditions, and to be consistent with the work we respond to directly [16]. We use the CGRA architecture as our reference architecture to be consistent with previous work, and since it's area and power consumption do not change when shared by multiple DSAs, thus the reference will not change when the number of DSAs to share is a dependent variable. The range of values we consider for α_{E2O} is consistent with published power and area carbon intensity data [25].

Finally, we also include the lifetime of an architecture relative to the reference CGRA as a parameter, as in certain environments a CGRA's reconfigurability *may* make an SoC less susceptible to obsolescence than DSAs. This is summarized in equation 3, where the new parameter L is the ratio of the reference CGRA architecture's lifetime to that of a test architecture, and P_{av} is the average power over the test architecture's lifespan, with power discounted each time the hardware is refreshed. For our analysis, we assume a 25% power-efficiency improvement every 2 years as is consistent with AMD's high performance desktop processors [26], and the reference (CGRA) lifespan to be 6 years, consistent with Azure's datacenter refresh schedule [8].

$$CF_{total} = C_a AL + C_p P_{av} \quad (3)$$

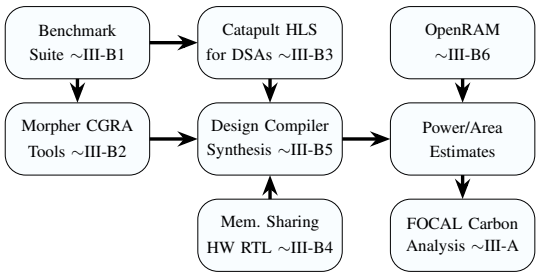


Fig. 4. The flow of our power and area estimation tools into our carbon analysis.

Security	AI/ML	Image Processing	DSP
AES Encrypt	2 Matrix Mult	DCT	FFT
ChaCha20 Encrypt	2d-Convolution	Histogram EQ	DFT
Blake2s Hash	Max Pooling	RGB to YUV	FIR
SHA-256 Hash	Batch Norm	Auto Focus	IIR

TABLE I
THE SET OF BENCHMARK KERNELS ANALYZED. FOUR KERNELS WERE CHOSEN FROM EACH OF FOUR APPLICATION DOMAINS.

B. Power and Area Estimation

Figure 4 shows our toolflow for analyzing the carbon footprint of each architecture under study.

1) Benchmarks: We assembled a comprehensive suite of 16 commonly accelerated benchmark kernels on which to base our instantiations of each architecture, clustered into groups of four kernels from each of four application domains. Table I lists our kernels by domain. This clustering allows for a wide variety of DSAs while also allowing for HCGRAs to be specialized for different application areas. These kernels are derived from a wide variety of open source repositories [27]–[36].

2) (H)CGRA Generation and Mapping: We chose our CGRA architecture to be consistent with prior work [16]. As depicted in Figure 3 we used an 8x8 CGRA composed of four, 4x4 CGRAs, each based on the HyCUBE architecture [20]. Each 4x4 CGRA is connected to 8 banks of SRAM, for a total of 32 banks. Each memory bank is an equal size, and this size is a parameter we vary in our analysis. An HCGRA was generated for each application domain using REVAMP [21], with this CGRA architecture as the base from which to remove hardware. Benchmark kernels were mapped on CGRA and HCGRA using Morpher [37]. If the HCGRA performed worse than the CGRA for a given mapping, we added resources back to the HCGRA by hand until it met the CGRA's performance, to ensure we were comparing architectures with equal performance.

3) Generating DSAs with HLS: We used the HLS tool Catapult to generate RTL for DSAs for our benchmarks. Catapult and Morpher were fed nearly identical C-language inputs. To create DSAs with equal performance to the CGRAs, we modulated factors of loop unrolling, pipeline initiation interval, clock frequency, and number of duplicate instances of the DSA for an algorithm. Area was used as the optimization target in Catapult.

4) Instantiating Low Overhead Memory Sharing Hardware: As discussed in Section II-B4, we implemented and tested an RTL description of low-overhead memory sharing hardware, as shown in Figure 2. For our carbon analysis, we implement this hardware with 32 banks of SRAM (consistent with the (H)CGRA).

5) Synthesis: We synthesized each architecture in Design Compiler for power and area estimates. We used the Nangate/Silvaco open cell library based on the 45nm FreePDK predictive silicon

Architecture	Av. Power (μW)	Av. Area (μm^2)
CGRA	$142,999$	$1,716,528$
HCGRA	$72,727$	$1,086,105$
Average DSA	$1,372$	$16,619$
Mem Sharing overhead	$3,372 + 222N_{dsa}$	$48,027 + 10603N_{dsa}$

TABLE II
AREA AND POWER CONSUMPTION ESTIMATES FOR THE COMPUTE HARDWARE FOR EACH ARCHITECTURE IN OUR ANALYSIS.

technology library for all synthesis [38], [39]. Without switching characteristics available for representative executions of each kernel for all architectures, our estimates rely on Design Compiler's default switching activity profile.

6) SRAM Compilation with OpenRAM: To compile SRAMs for power and area estimation we used OpenRAM [40]. Each memory bank was compiled with a single read/write port, and with a word size of 32 bits. We compiled memories using the same 45nm technology library as used for synthesizing the compute hardware. For our analysis, we assume that each kernel accesses its lowest level memory twice per cycle on average, which is consistent with the schedules produced by Catapult.

IV. RESULTS

A. Compute Area and Power Consumption

Table II summarizes our area and power estimates for each architecture. Compared to CGRA, the average HCGRA requires 36.7% less area, and 49.1% less power, consistent with existing work [21]–[23]. The HCGRA generated for security kernels had significantly higher overheads than for the other three domains, as it could not meet the CGRA's performance with REVAMP's changes to the configuration register files, so its register files were left unchanged.

The average DSA with equivalent performance to the (H)CGRA is over 100x smaller and less power consuming than the CGRA architecture. Though this result may be surprising at first, it makes sense when considering that an 8x8 CGRA requires 64 configuration register files, 64 full ALUs, 64 crossbar routers, and a lot of interconnect. Furthermore, because each PE in the CGRA can only execute one operation per cycle, pipeline initiation intervals are higher, indicating the CGRA is less efficient with its resources. Operations which require no delay and no hardware in a DSA, like shifting by a constant, require an entire cycle and PE in a CGRA, and fixed-point operations requires a slow, software-like implementation. On the other hand, to match the CGRA's performance, an average DSA only needs a few operators and registers, simple state logic, and significantly less interconnect. As such, the large difference in area and power between the CGRA and DSA is expected.

The overhead of the memory sharing hardware for the FFSM architecture is significant compared to the size of a DSA, but has a small fraction of the overhead of CGRA. The specific overhead depends on the number of DSAs the hardware was configured to interface with, as each new interface port requires more interconnect and logic to be instantiated in the sharing hardware.

One way to quantify the sustainability of a sharing architecture compared to a sea of DSAs, is to specify what we call a *kernel sharing threshold*. A sharing architecture's *kernel sharing threshold* is the number of accelerated kernels which must share hardware (be it a CGRA, an HCGRA, or just memory) for that architecture to reduce area enough to be more carbon-efficient than building a sea of DSAs to execute those kernels.

When just considering compute (ignoring cache/scratchpad memories), neither CGRA nor HCGRA provides a sustainability benefit with a reasonable kernel sharing threshold. Even in cases highly dominated by embodied emissions ($\alpha_{cgra} = 0.95$), the CGRA would need to be shared by 109 kernels, and the HCGRA by 68 kernels. These numbers balloon to 513 and 272 in operationally dominated conditions. It is unrealistic to expect a CGRA to be shared by 109 kernels, or an HCGRA to be shared by 68, without a performance cost, as this many kernels cannot share a single CGRA without timing conflicts. As such, to see how a sharing architecture can decrease emissions, we need to look at *memory*.

Bank Size	Leakage Power (μW)	Dynamic Power per Access (μW)	Area (μm^2)
128B	1.4	36.5	$9,030$
256B	2.4	63.1	$9,243$
512B	4.7	116.1	$14,858$
1kB	8.9	210.1	$19,460$
2kB	17.7	410.7	$34,469$
4kB	34.6	788.8	$64,790$
8kB	69.1	1575.4	$118,617$
16kB	136.7	3134.1	$240,817$
32kB	273.1	6352.8	$447,698$

TABLE III
THE AREA AND POWER CONSUMPTION ESTIMATES FOR DIFFERENT SIZED BANKS OF SRAM IN OUR ANALYSIS.

B. SRAM Area and Power Consumption

Table III details the power and area estimates for SRAM banks of different sizes, compiled using *OpenRAM*. Generally, the area and power consumption of a bank of SRAM are linearly proportional to it's size. For smaller banks, however, the hardware the bank requires outside of it's array of bit storage (e.g. sense amplifiers) is more significant, especially as a proportion of the bank's area.

Comparing the memory banks to the compute hardware, it is clear that SRAM contributes a significant proportion the area and power consumption of any hardware acceleration, especially DSAs.

Because the smallest bank size OpenRAM could compile was 128B, and our CGRA and HCGRA architectures use 32 banks, we set our smallest considered memory size to 4kB (32 banks of 128B). At smaller memory sizes, the ratio between our DSA power and area estimates and our memory power and area estimates is more consistent with existing work [18], [19]. For this reason, we consider these low-memory cases as more realistic. However, we consider kernel memory capacity requirements up to 256kB, to be consistent the with prior work we directly respond to [16].

C. Mobile & Embedded SoCs ($L = 1$)

Figure 5 plots the *kernel sharing threshold*, the number of application kernels which must share a CGRA, HCGRA, or FFSM architecture in order to emit less carbon than a sea of DSAs for those kernels.

In these graphs, a sea of DSAs with, for example, a DSA for each of 9 kernels, would have a horizontal line at $y = 9$, where lower is more sustainable. As such, if 9 kernels can share hardware without a performance penalty, then all sharing architectures can provide a sustainability win over a sea of DSAs at $\alpha_{cgra} = 0.8$ if the average kernel requires 256kB of memory, but only FFSM can if the kernels require 4kB on average (third panel in the top row of Figure 5).

The top row of graphs assumes reconfigurable acceleration cannot extend the lifetime of the device ($L = 1$), as is the case for mobile and embedded SoCs, where applications change less frequently, and the lifespan of the device is more dependent on wear and tear than on the performance of the processor. In embodied dominant cases, at large memory sizes, all of the memory sharing architectures approach a minimum value of 2 kernels. 2 is the minimum because each

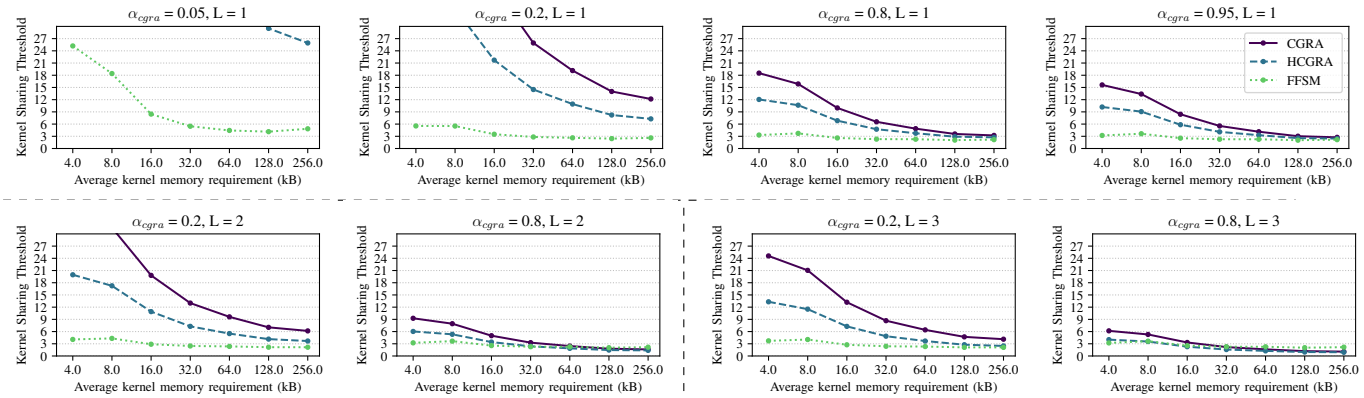


Fig. 5. The number of kernels which must share hardware for a given architecture to have a lower carbon footprint than a sea of DSAs at different operating conditions (α_{CGRA}), lifespan ratios ($L = L_{CGRA}/L_{DSA}$), and average kernel memory capacity requirements. Each architecture is allocated twice the memory required for an average kernel. Lower is better.

memory sharing architecture is allocated a pool of memory twice the size of the average kernel memory capacity requirement (the x-axis values; see Section IV-F for further discussion of this ratio). Thus any sharing is more sustainable than a sea of DSAs in these high-memory, embodied dominant cases, as each kernel is active only a small proportion of time, so 2 kernels can share easily.

At smaller memory sizes and in operationally dominated cases, however, the kernel sharing threshold for (H)CGRA quickly rises. As such, it is clear that (H)CGRA can only provide a sustainability benefit over a sea of DSAs in mobile/embedded contexts under ideal conditions, with dominant embodied emissions and high memory requirements, and even in these cases, the FFSM architecture performs marginally better.

The fixed-function compute with shared memory (FFSM) architecture is the clear winner. The kernel sharing threshold for FFSM increases only a small amount over it's asymptotic minimum for all but the most operationally dominated case. For $\alpha_{cgra} = 0.8$ with a small average kernel memory size like 4kB, the FFSM architecture's kernel sharing threshold is 3.6X and 5.6X smaller than than the critical threshold for HCGRA and CGRA respectively. In the operationally dominated cases, the threshold is always more than 2.8X and 4.7X smaller than HCGRA and CGRA. There is no measured case for $L = 1$ where this architecture is less sustainable than either the HCGRA or the CGRA.

D. Datacenter SoCs ($L > 1$)

In the datacenter, frequent changes in applications mean that reconfigurable acceleration may, theoretically, increase the lifetime of a device. In line with this, the bottom row of graphs considers cases where an (H)CGRA's lifetime is 2x ($L = 2$, left) and 3x ($L = 3$, right) that of a sea of DSAs or FFSM architecture.

Throughout the bottom row, CGRA and HCGRA put up substantially better showings than the top row. Once again we see that CGRA and HCGRA perform best in embodied dominant conditions with high memory requirements, even very marginally beating FFSM at a couple of data points in these cases.

However, in the datacenter, accelerator emissions are not embodied dominant. The emissions of Google's TPUs, for example, are between 70% and 90% operational [41]. In more realistic, operationally dominated cases, FFSM beats CGRA (HCGRA) at all data points, reducing kernel sharing threshold by up to 9.1x (4.9x)

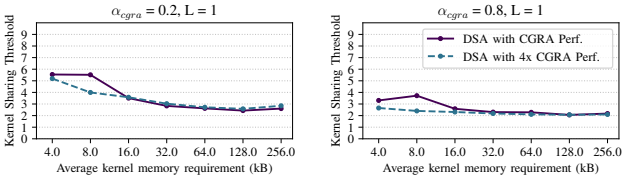


Fig. 6. FFSM kernel sharing threshold with different levels of DSA performance. Lower is better.

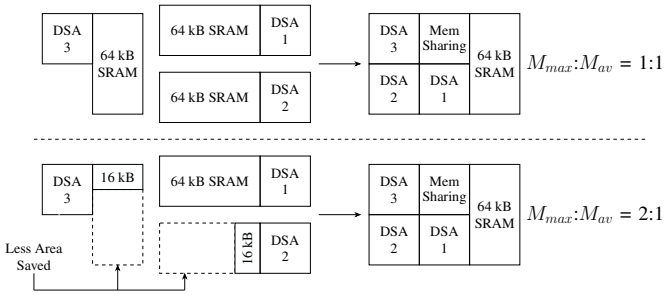


Fig. 7. The ratio between maximum and average DSA memory requirements affect the ability of a memory sharing architecture to reduce area.

E. DSAs with Higher Performance

Because we consider DSAs with performance equivalent to that of a CGRA, we are necessarily considering relatively small DSAs. To examine how this impacts our results, we used *Catapult* to instantiate and synthesize versions of each DSA with 4 times the performance of the CGRA. To do this, we used combinations of inner loop unrolling, shrinking pipeline iteration intervals, increasing clock frequency, and instantiating more instances of our DSAs. The average area and power consumption respectively for these higher performance DSAs were 1.83x and 3.23x those of the DSAs with CGRA levels of performance. Figure 6 illustrates the small effect that this change has on the sustainability of the FFSM architecture (note the smaller y-axis). Though the area and power consumptions of the compute hardware rise relative to the memory hardware, they also rise relative to the overhead of the hardware instantiated for sharing memory. These two effects mostly cancel.

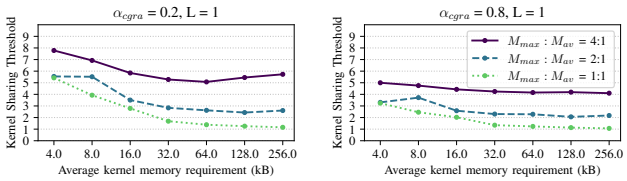


Fig. 8. FFSM kernel sharing threshold for different ratios between the memory allocated for the shared pool and the average memory required by a kernel. Lower is better.

F. Maximum to Average Memory Ratio

A contemporary SoC may contain many tens of DSAs [17], and it is infeasible from a floor planning and a time sharing perspective for this many kernels to share a single pool of lowest level memory. As such, it is important to consider how to best cluster these kernels.

The decrease in area achievable by a memory sharing architecture depends significantly on the ratio between the average memory required by a kernel and the amount of memory which must be allocated in the shared memory pool. Figure 7 visualizes this importance. To examine this effect quantitatively, we consider the FFSM architecture with different ratios between size of the FFSM memory pool and the average memory size required by the shared kernels. For example, for a ratio of 4:1, and an average kernel memory requirement of 4kB, 16kB of SRAM will be allocated in the sharing architecture's pool. The results of this evaluation are plotted in Figure 8. All other data presented throughout this work uses a 2:1 ratio between the size of the shared memory pool and the average DSA memory requirement.

Changing between different ratios shifts the critical threshold along the y axis. As expected, decreasing the ratio to 1:1 decreases the number of kernels which must share, and increasing the ratio to 4:1 increases the threshold.

V. Discussion

In our analysis we do not consider concurrency. Though a reconfigurable fabric's performance is halved when shared for two kernels (each uses 2 4x4 subarrays of the 8x8 array), the more specialized architectures handle concurrency more gracefully. Concurrency has little effect on a sea of DSAs, and FFSM performance is only diminished when multiple accelerators try to access the same memory bank in the same cycle, which often be designed around (for example, by interleaving shared data into two memory banks, and having two concurrent accelerators both switch back and forth).

Memory sharing architectures have decreased data movement costs compared to a sea of DSAs. Whereas a sea of DSAs must transmit data through main memory, a higher level cache, or from peer-to-peer [18], [42], sharing architectures will often not have to move data at all for multiple kernels to operate on the data. We do not account for this when equalizing performance.

For the CGRA and HCGRA architectures, the mappings for each kernel must be stored somewhere. In this work, we have assumed that this storage cost is minimal and that reconfiguration time for the CGRA architectures is negligible. In reality, storing the mappings on chip would endure an area cost, and storing them off chip would lead to a performance penalty when reconfiguring.

VI. Conclusion

In this work we explored how sharing private-local memory amongst many DSAs can improve the sustainability of hardware acceleration for systems on chip. We highlighted that using fixed-function compute with shared memory can provide a sustainability win in many different scenarios.

REFERENCES

[1] C. Freitag, M. Berners-Lee, K. Widdicks, B. Knowles, G. S. Blair, and A. Friday, "The real climate and transformative impact of ict: A critique of estimates, trends, and regulations," *Patterns*, vol. 2, no. 9, p. 100340, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666389921001884

[2] S. R. Group. (2024) Hyperscale data centers hit the thousand mark; total capacity is doubling every four years. [Online]. Available: https://www.srgresearch.com/articles/hyperscale-data-centers-hit-the-thousand-mark-total-capacity-is-doubling-every-four-years

[3] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang, C. Bai, M. Gschwind, A. Gupta, M. Ott, A. Melnikov, S. Candido, D. Brooks, G. Chauhan, B. Lee, H.-H. Lee, B. Akyildiz, M. Balandat, J. Spisak, R. Jain, M. Rabbat, and K. Hazelwood, "Sustainable ai: Environmental implications, challenges and opportunities," in *Proceedings of Machine Learning and Systems*, D. Marculescu, Y. Chi, and C. Wu, Eds., vol. 4, 2022, pp. 795–813. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2022/file/462211f67c7d858f663355eff93b745e-Paper.pdf

[4] S. Sudhakar, V. Sze, and S. Karaman, "Data centers on wheels: Emissions from computing onboard autonomous vehicles," *IEEE Micro*, vol. 43, no. 1, pp. 29–39, 2023.

[5] D. Patterson, "The carbon footprint of machine learning training will plateau, then shrink," Feb. 2022. [Online]. Available: http://dx.doi.org/10.36227/techrxiv.19139645.v1

[6] TechInsights, "Techinsights launches the bom data carbon emissions modules," 2023. [Online]. Available: https://www.techinsights.com/sustainability/techinsights-launches-bom-data-carbon-emissions-modules

[7] Apple, "Product environmental report iphone 15 pro and iphone 15 pro max," Apple, Tech. Rep., 2023. [Online]. Available: https://www.apple.com/environment/pdf/products/iphone/iPhone_15_Pro_and_iPhone_15_Pro_Max_Sept2023.pdf

[8] J. Wang, D. S. Berger, F. Kazhamiaka, C. Irvene, C. Zhang, E. Choukse, K. Frost, R. Fonseca, B. Warrier, C. Bansal, J. Stern, R. Bianchini, and A. Sriraman, "Designing cloud servers for lower carbon," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 452–470.

[9] D. Kline, N. Parshook, X. Ge, E. Brunvand, R. Melhem, P. K. Chrysanthis, and A. K. Jones, "Greenchip: A tool for evaluating holistic sustainability of modern computing systems," *Sustainable Computing: Informatics and Systems*, vol. 22, pp. 322–332, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2210537917300823

[10] U. Gupta, Y. G. Kim, S. Lee, J. Tse, H.-H. S. Lee, G.-Y. Wei, D. Brooks, and C.-J. Wu, "Chasing carbon: The elusive environmental footprint of computing," *IEEE Micro*, vol. 42, no. 4, p. 37–47, Jul. 2022. [Online]. Available: https://doi.org/10.1109/MM.2022.3163226

[11] L. Eeckhout, "Focal: A first-order carbon model to assess processor sustainability," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 401–415. [Online]. Available: https://doi.org/10.1145/3620665.3640415

[12] ——, "Kaya for computer architects: Toward sustainable computer systems," *IEEE Micro*, vol. 43, no. 1, pp. 9–18, 2023.

[13] V. J. Reddi. (2018) Mobile socs: The wild west of domain specific architectures. [Online]. Available: https://www.sigarch.org/mobile-socs/

[14] E. Brunvand, D. Kline, and A. K. Jones, "Dark silicon considered harmful: A case for truly green computing," in *2018 Ninth International Green and Sustainable Computing Conference (IGSC)*, 2018, pp. 1–8.

[15] H. Esmaeilzadeh, E. Blem, R. S. Amant, K. Sankaralingam, and D. Burger, "Dark silicon and the end of multicore scaling," in *2011 38th Annual International Symposium on Computer Architecture (ISCA)*, 2011, pp. 365–376.

[16] P. Dangi, T. K. Bandara, S. Sheikhpour, T. Mitra, and L. Eeckhout, "Sustainable hardware specialization," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '24. New York, NY, USA: Association for Computing Machinery, 2024.

[17] Y. S. Shao, B. Reagan, G.-Y. Wei, and D. Brooks, "Retrospective: Aladdin: a pre-rtl, power-performance accelerator simulator enabling large design space exploration of customized architectures," in *ISCA@50 25-Year Retrospective: 1996-2020*, J. F. Martínez and L. K. John, Eds. ACM SIGARCH and IEEE TCCA, 2023. [Online]. Available: https://bit.ly/isca50_retrospective

[18] D. Giri, P. Mantovani, and L. P. Carloni, "Accelerators and coherence: An soc perspective," *IEEE Micro*, vol. 38, no. 6, pp. 36–45, 2018.

[19] M. J. Lyons, M. Hempstead, G.-Y. Wei, and D. Brooks, "The accelerator store: A shared memory framework for accelerator-based systems," *ACM Trans. Archit. Code Optim.*, vol. 8, no. 4, Jan. 2012. [Online]. Available: https://doi.org/10.1145/2086696.2086727

[20] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, "Hycube: A cgra with reconfigurable single-cycle multi-hop interconnect," in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2017, pp. 1–6.

[21] T. K. Bandara, D. Wijerathne, T. Mitra, and L.-S. Peh, "Revamp: a systematic framework for heterogeneous cgra realization," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 918–932. [Online]. Available: https://doi.org/10.1145/3503222.3507772

[22] J. Li, Y. Qiu, G. Zhu, Q. Zhu, W. Yin, and L. Wang, "Thram: A template-based heterogeneous cgra modeling framework supporting fast dse," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–5.

[23] Y. Dai, J. Li, Q. Zhu, Y. Qiu, Y. Hu, W. Yin, and L. Wang, "Heta: A heterogeneous temporal cgra modeling and design space exploration via bayesian optimization," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 3, pp. 505–518, 2024.

[24] U. Gupta, M. Elgamal, G. Hills, G.-Y. Wei, H.-H. S. Lee, D. Brooks, and C.-J. Wu, "Act: designing sustainable computer systems with an architectural carbon modeling tool," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 784–799. [Online]. Available: https://doi.org/10.1145/3470496.3527408

[25] L. Boakes, M. Garcia Bardon, V. Schellekens, I.-Y. Liu, B. Vanhouche, G. Mirabelli, F. Sebaai, L. Van Winckel, E. Gallagher, C. Rolin, and L. Ragnarsson, "Cradle-to-gate life cycle assessment of cmos logic technologies," in *2023 International Electron Devices Meeting (IEDM)*, 2023, pp. 1–4.

[26] AMD. (2022, Jun) Amd details strategy to drive next phase of growth across $300 billion market for high-performance and adaptive computing solutions. [Online]. Available: https://ir.amd.com/news-events/press-releases/detail/1078/amd-details-strategy-to-drive-next-phase-of-growth-across-300-billion-market-for-high-performance-and-adaptive-computing-solutions

[27] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to gpu codes," in *2012 Innovative Parallel Computing (InPar)*, 2012, pp. 1–10.

[28] S. Bakhurin, R. C. Schmidt, and S. Royz, "Libdspl2," 2018. [Online]. Available: https://github.com/Dsplib/libdspl-2.0

[29] J. M. de la Rosa, "Iir_filters," 2018. [Online]. Available: https://github.com/jmarcor/IIR_filters

[30] J. Wilczek, "fir-simd," 2022. [Online]. Available: https://github.com/JanWilczek/fir-simd

[31] M. Kohn, "simd_examples," 2018. [Online]. Available: https://github.com/mikeakohn/simd_examples

[32] L. Project, "Libtomcrypt," 2010. [Online]. Available: https://github.com/libtom/libtomcrypt

[33] Kokke, "tiny-aes-c," 2012. [Online]. Available: https://github.com/kokke/tiny-AES-c

[34] T. Roberts, M. Slaney, D. P. Bouras, and E. Condes, "fix_fft," 2017. [Online]. Available: https://docs.arduino.cc/libraries/fix_fft/

[35] P. Ammon, "Libdivide," 2010, accessed 2025. [Online]. Available: https://libdivide.com

[36] S. Barrett, "stb," 2014. [Online]. Available: https://github.com/nothings/stb

[37] D. Wijerathne, Z. Li, M. Karunaratne, T. Mitra, and L.-S. Peh, "Morpher: An open-source integrated compilation and simulation framework for cgra," in *2022 Workshop on Open-Source EDA Technology (WOSET), Article No. 12*, 2022, p. 5.

[38] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, and R. Jenkal, "Freepdk: An open-source variation-aware design kit," in *2007 IEEE International Conference on Microelectronic Systems Education (MSE'07)*, 2007, pp. 173–174.

[39] S. I. I. (SI2). Open cell and free pdk libraries. [Online]. Available: https://si2.org/open-cell-and-free-pdk-libraries/

[40] M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, "Openram: An open-source memory compiler," in *2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016, pp. 1–6.

[41] I. Schneider, H. Xu, S. Benecke, D. Patterson, K. Huang, P. Ranganathan, and C. Elsworth, "Life-cycle emissions of ai hardware: A cradle-to-grave approach and generational trends," 2025. [Online]. Available: https://arxiv.org/abs/2502.01671

[42] K.-L. Chiu, D. Giri, L. Piccolboni, and L. P. Carloni, "An analysis of accelerator data-transfer modes in noc-based soc architectures," in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*, 2023, pp. 1–7.